

Lahore University of Management Sciences
Department of Artificial Intelligence

AI651 – Deep Learning for Space, Time and Graphs

Programming Assignment 1

Hasan Amjad
Roll No. 25280042

GitHub repository:

<https://github.com/USERNAME/REPO> [TODO: replace with your public repo link]

October 3, 2026

Contents of submission: this report (PDF and $\text{\LaTeX}$ source); the executed PA1_PRESET=full Task 1 notebook; each numbered figure as a PDF; Task 2 code in the repository.

Task 1 – Forecasting Across Heterogeneous Sensors

Run configuration. Preset full; data seed 0; model seed 0; device mps (Apple GPU); eligible train/validation/test origins 650/105/105. All numbers below come from this single full run. Unless stated otherwise, errors are **validation** RMSE in m/s^2 ; “established” = Stations 1–3, “held-out” = Station 4.

Response 1A – Predicted vs. measured ordering

Prediction (written before running Output 1.1).

- **Established stations** (best $\rightarrow$ worst): Raw Attention $>$ Period-routed ridge $>$ Shared ridge $>$ Sensor-specific ridge.
- **Held-out Station 4** (best $\rightarrow$ worst): Raw Attention $>$ Period-routed ridge $>$ Shared ridge; Sensor-specific ridge cannot forecast Station 4.
- **Reasoning.** Raw Attention is a neural network that builds its own mixing rule from each context, so it should handle changing speed and also forecast Station 4. Period-routed ridge picks a formula fitted to the estimated speed, so it also handles changing speed and Station 4, but relies on a hand-designed router. Shared ridge applies one formula everywhere: it cannot adapt to speed, but it still forecasts Station 4. Sensor-specific ridge uses a fixed formula per station, so it cannot adapt to changing speed, and it has no formula for Station 4.

Table 1: Output 1.1 – validation RMSE (m/s^2) by population and product.

Model	Established (Stations 1–3)				Held-out (Station 4)			
	All	A	B	C	All	A	B	C
Shared ridge	0.767	0.805	0.854	0.621	0.876	0.909	0.982	0.713
Sensor-specific ridge	0.768	0.804	0.852	0.630	n/a	n/a	n/a	n/a
Period-routed ridge	0.166	0.181	0.164	0.152	0.173	0.184	0.182	0.150
Raw Attention	0.436	0.412	0.462	0.433	0.659	0.677	0.619	0.678

Measurement (Output 1.1).

Key differences, quantified.

- Period-routed ridge has the lowest error everywhere: 0.166 on established stations, **78% lower** than shared ridge (0.767) and **62% lower** than Raw Attention (0.436). It is also uniform across products (0.15–0.18).
- Shared and sensor-specific ridge are practically identical (0.767 vs. 0.768): giving each station its own matrix buys nothing.
- Generalisation to the unseen station: period-routed ridge degrades by only 4% (0.166 $\rightarrow$ 0.173), shared ridge by 14% (0.767 $\rightarrow$ 0.876), and Raw Attention by **51%** (0.436 $\rightarrow$ 0.659).

Revised ordering and explanation. Measured ordering, both populations: **Period-routed ridge** $>$ **Raw Attention** $>$ **Shared ridge** $\approx$ **Sensor-specific ridge** (the latter unavailable on Station 4). My prediction got the bottom of the ranking right but the top wrong.

- *Shared $\approx$ sensor-specific:* the forecasting rule depends on operating pace, not on station identity, so splitting the data by station removes training data without removing the cross-pace compromise.
- *Period-routed ridge wins:* it conditions on exactly the quantity that changes (the period) and, within a pace band, continuation really is close to linear. A human supplied the right variable, so a small closed-form model suffices.
- *Raw Attention is second:* its context-built mixing beats one fixed matrix (0.436 vs. 0.767), but it must *learn* the pace dependence from the data while the slow drift dominates the raw input

and every position pair is scored separately. Its large held-out gap suggests it partly fitted station-specific details instead of the pace structure.

Response 1B – Why the forecasting rule must depend on context

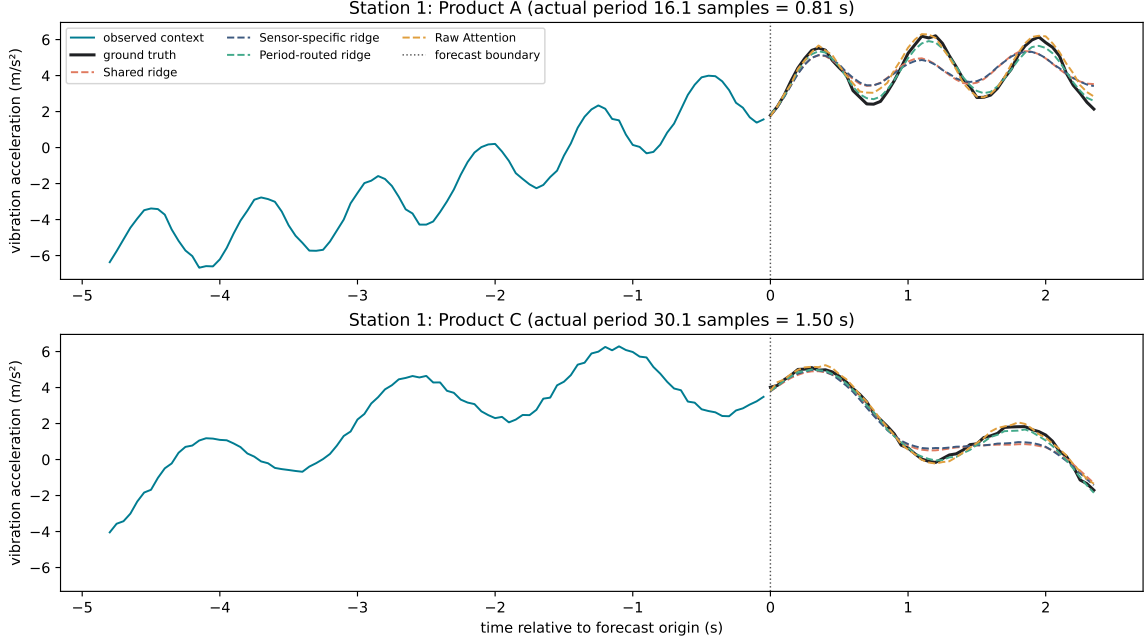


Figure 1: Output 1.2 – Station 1 under Product A ($P \approx 16.1$) and Product C ($P \approx 30.1$).

The single matrix $\widehat{W}$ in Eq. (8) is fitted by least squares over *all* training windows, which mix periods from 14 to 36 samples. A 16-sample cycle and a 30-sample cycle need different phase advances over the same 48-step horizon, so one fixed operator can only return their least-squares compromise. In Figure 1 this compromise is visible as **amplitude shrinkage and phase lag**: for Product A the shared and sensor-specific ridge forecasts have visibly flattened peaks and fall behind the true oscillation in the second half of the horizon (window RMSE 0.779 and 0.787), while period-routed ridge, which uses a matrix fitted only to similar paces, tracks it (0.232). Averaging continuations whose phases disagree partly cancels them, which is why the compromise forecast is damped rather than simply shifted.

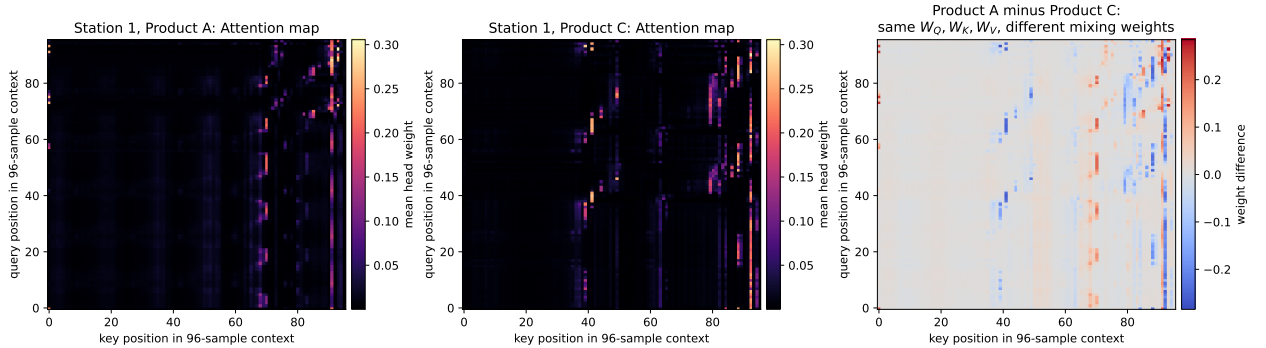


Figure 2: Output 1.3 – Attention maps for two Station 1 contexts and their difference.

Attention uses the *same* learned W_Q, W_K, W_V for both contexts, but its scores $QK^\top/\sqrt{d_h}$ are computed from the current window. Different inputs therefore give different weight matrices: in Figure 2 the Product A and Product C maps concentrate on different key positions, with a mean

absolute weight difference of 0.013 and a maximum of 0.284. Each context thus receives its own value-mixing operation, whereas ridge applies the identical $\widehat{W}$ to every window. (This only shows that the mixing is context-dependent; it is not evidence that the attention weights have recovered the physical period.)

Response 2 – Separating slow structure from repetition

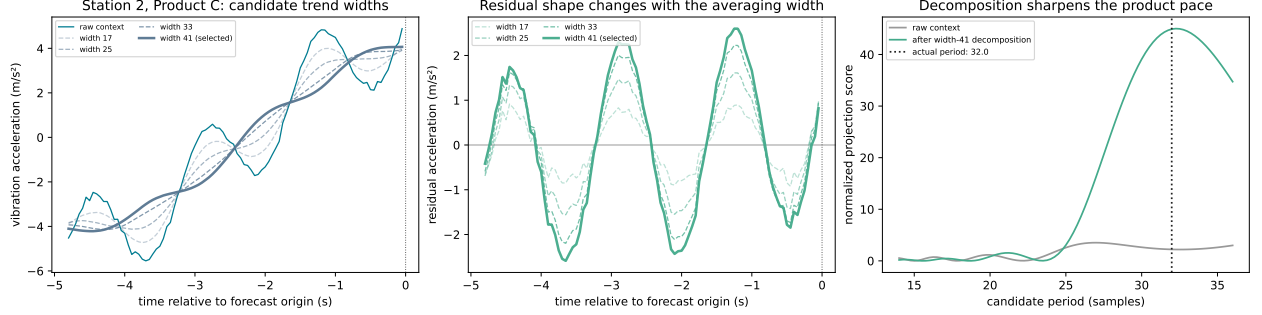


Figure 3: Output 2.1 – trend and remainder for candidate widths (Station 2, Product C).

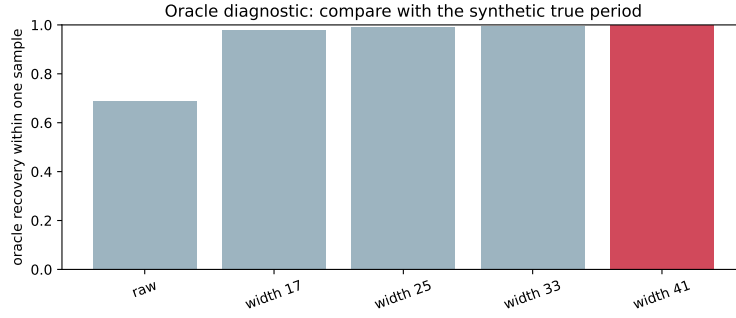


Figure 4: Output 2.2 – oracle period recovery (training data only).

Table 2: Outputs 2.1–2.2 – effect of the averaging width m .

	raw	$m = 17$	$m = 25$	$m = 33$	$m = 41$
Remainder std (m/s^2)	–	0.578	1.012	1.372	1.572
Oracle recovery within one sample	0.687	0.980	0.989	0.993	0.998
Median period error (samples)	0.525	0.157	0.122	0.142	0.216

Table 3: Output 2.3 – decomposition ablation, validation RMSE (m/s^2).

Model	Established	Held-out	Est. A / B / C	Held-out A / B / C
Raw Attention	0.436	0.659	0.412 / 0.462 / 0.433	0.677 / 0.619 / 0.678
Attention + decomposition	0.324	0.486	0.302 / 0.375 / 0.289	0.428 / 0.556 / 0.464

- **Selected width $m = 41$.** Visually (Fig. 3), narrow widths let the trend follow part of the vibration: at $m = 17$ the trend wiggles and the remainder is weak (std 0.578). As m grows the trend becomes a smooth ramp and the remainder becomes a clean, full-amplitude oscillation (std 1.572 at $m = 41$); after decomposition the period score peaks sharply at the true period (32), while the raw context gives almost no peak. Empirically, oracle period recovery rises from 0.687 (raw) to 0.998 ($m = 41$), and in Output 2.3 adding decomposition to Attention lowers

validation RMSE by **26%** on established stations ($0.436 \rightarrow 0.324$) and by **26%** on Station 4 ($0.659 \rightarrow 0.486$), improving every product.

- **Why a moving average suits this world.** The drift varies on a 240-sample scale while the vibration has periods of 14–36 samples. A 41-sample window is longer than every vibration period, so it averages each cycle to approximately zero, yet it is much shorter than 240, so the drift passes into the trend almost unchanged. The training-only oracle diagnostic is a sound way to choose m because it directly measures the property we want (does the remainder expose the true repetition?) using information available only in the synthetic training data, without touching validation or test data. Note that the selection criterion matters: $m = 41$ maximises recovery within one sample, while $m = 25$ has the smallest median period error.
- **Where it fails / alternative.** A fixed moving average fails at an *abrupt level shift* (e.g. a product changeover or impact inside the window): it smears the step over 41 samples and leaks a spurious transient into the remainder. It would also fail if a vibration period exceeded the window. An alternative is **STL/LOESS decomposition**, which fits local polynomials and assumes a locally smooth (curved) trend rather than a locally constant one, at the cost of more computation and tuning.

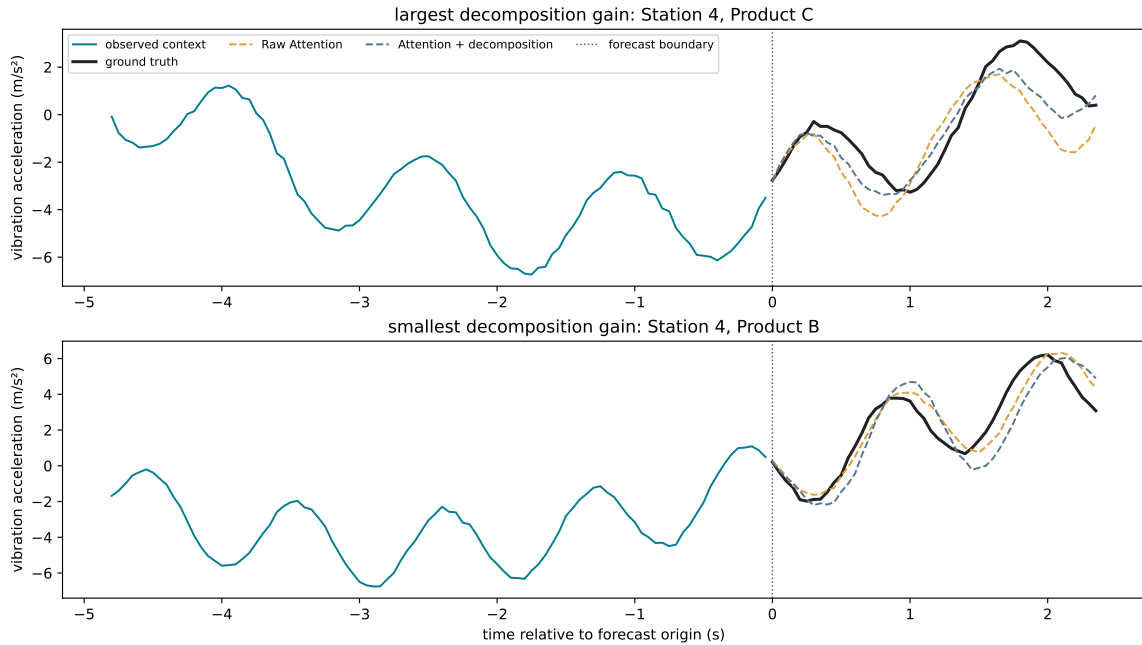


Figure 5: Output 2.3 – largest and smallest decomposition gains (Station 4).

Response 3 – Mixing by temporal delay

Implementation check: `aggregate_delays` reproduces the worked example (35, 15, 25, 25, so $z_0 = 35$ and the shift runs the right way) and passes the gradient check; Output 3.1 confirms `delay_scores` against a direct circular correlation (4, -4, 4, -4).

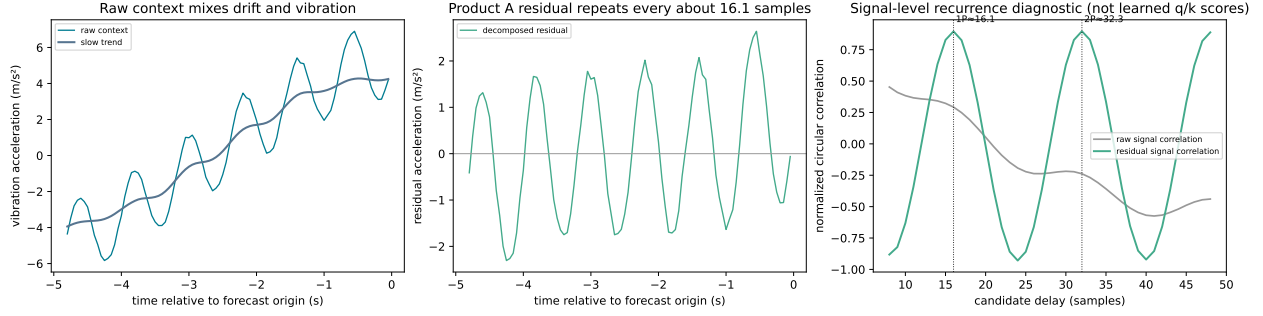


Figure 6: Output 3.2 – signal-level recurrence diagnostic (Station 2, Product A, $P \approx 16.1$). This is a correlation computed directly from the signal, not the model’s learned q/k scores.

- **Strongest lags.** The residual correlation peaks at lag **16** (0.896) and lag **32** (0.897), i.e. at the run’s period $P \approx 16.1$ and its multiple $2P \approx 32.3$; the troughs at about 8, 24 and 40 samples are the half-period offsets, where peaks align with troughs.
- **Why decomposition cleans the evidence.** In the raw signal the correlation simply decays from about 0.45 at lag 8 to -0.55 at lag 40, with no peak at P : the slow ramp is similar to itself at every small shift and dominates the score. Removing the trend leaves an almost pure oscillation, so peaks align only at multiples of P . This supports the delay mixer’s inductive bias: if useful information sits at a few recurring displacements, scoring L delays (instead of L^2 position pairs) and aggregating shifted copies is the natural mechanism, and it works best on the decomposed stream. Consistently, in Output 4.1 the delay mixer alone already halves Raw Attention’s error ($0.436 \rightarrow 0.214$).
- **Where delay mixing is the wrong bias.** An *isolated transient*, such as a single impact from a loose part, has no recurring displacement. The delay mixer would either average it away or copy it forward at the selected lags as false “echoes”, because it can only reuse shifted copies of the context. Similarly, if the period drifts quickly within one context (machine run-up), no small fixed set of lags aligns the whole window.

Response 4 – Comparing designs and deploying

(a) **Prediction (written before Output 4.1).** I expect decomposition to help most, since it splits off the slow trend with a moving average and sends it to a simple linear head, leaving the encoder only the faster vibration.

Table 4: Validation error by model. RMSE in m/s^2 , MSE in $(\text{m/s}^2)^2$.

Model	Established		Held-out		Params	Fit (s)
	RMSE	MSE	RMSE	MSE		
Shared ridge	0.767	0.588	0.876	0.767	4,608	0.001
Sensor-specific ridge	0.768	0.590	n/a	n/a	13,824	0.001
Period-routed ridge	0.166	0.028	0.173	0.030	59,904	0.002
Raw Attention	0.436	0.190	0.659	0.434	368,368	206
Attention + decomposition	0.324	0.105	0.486	0.236	373,024	292
Raw delay mixer	0.214	0.046	0.338	0.114	368,370	776
Autoformer-inspired	0.197	0.039	0.244	0.059	373,026	852

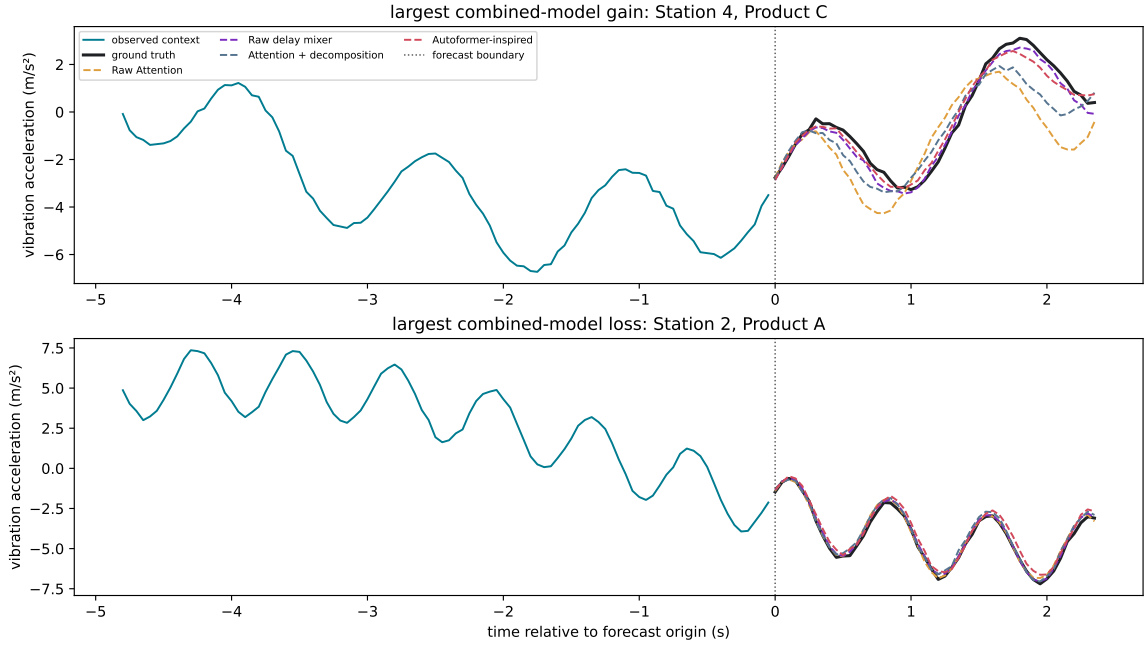


Figure 7: Output 4.1 – contrasting forecast cases for the four neural configurations.

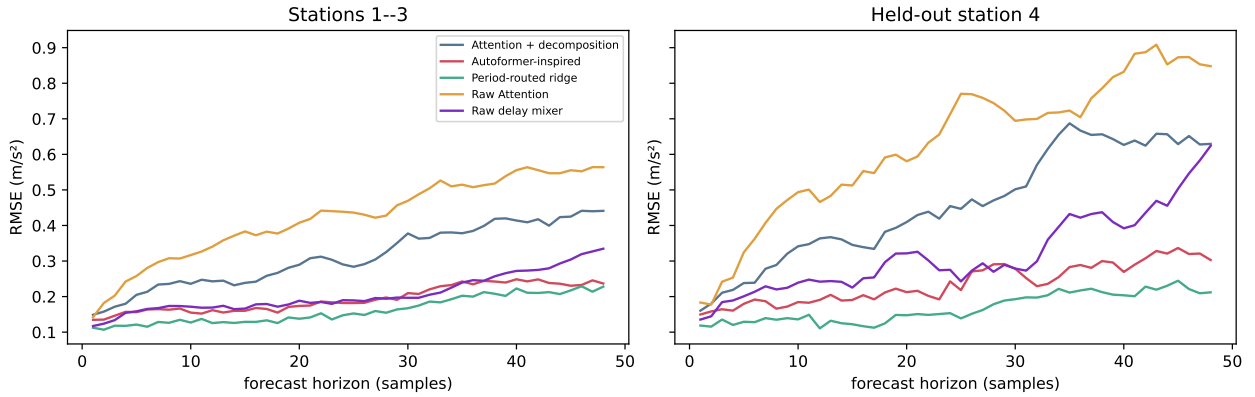


Figure 8: Output 4.2 – validation RMSE across the 48-step horizon.

(b) Validation results (Outputs 4.1–4.2).

Interpretation (≤ 200 words). On established stations, decomposition alone lowers Raw Attention’s RMSE from 0.436 to 0.324 (−26%), delay mixing alone to 0.214 (−51%), and both together to 0.197 (−55%). The gains are **sub-additive**: the individual RMSE reductions sum to 0.334, but the combined reduction is 0.239 (in MSE: 0.229 summed vs. 0.151 combined). This fits the hypothesis that both switches address overlapping drift-plus-repetition problems. On held-out Station 4 the overlap is smaller: adding decomposition to the delay mixer still cuts RMSE from 0.338 to 0.244 (−28%), versus only −8% on established stations. Contrary to my prediction, delay mixing was the stronger single upgrade overall; my prediction concerned drift-dominated contexts, which this aggregate table does not isolate. Across the horizon, all errors are similar at step 1 (0.11–0.15) and grow with lead time. Autoformer-inspired grows least among neural models (0.135 $\rightarrow$ 0.237), Raw Attention most (0.142 $\rightarrow$ 0.564), and period-routed ridge stays lowest throughout (0.112 $\rightarrow$ 0.228). Yet the hand-routed linear model still beats every neural variant.

(c) Deployment choices (fixed before Output 4.3).

- **Established stations: Period-routed ridge.** Lowest validation RMSE (0.166 vs. 0.197 for the best neural model), about $6\times$ fewer parameters (59,904 vs. 373,026), and essentially instant fitting (0.002 s vs. 852 s).
- **Held-out Station 4: Period-routed ridge.** Lowest held-out validation RMSE (0.173 vs. 0.244), with the same cost advantage; it routes by estimated pace rather than station identity, so it applies directly to an unseen station.

Table 5: Output 4.3 – selected models, validation and test.

Population	Model	Val. RMSE	Val. MSE	Test RMSE	Test MSE
Established	Period-routed ridge	0.166	0.028	0.161	0.026
Station 4	Period-routed ridge	0.173	0.030	0.183	0.033

Test errors stay close to validation (0.161 and 0.183), so the validation-based choice holds on unseen data. On the central question, the right inductive bias (conditioning on pace) mattered more here than additional model flexibility.

Task 2 – Leaderboard Challenge (Autoformer)

Data exploration and periodic structure

[TODO: ACF, recovered periods, heavy tails, transforms]

Validation protocol

[TODO: chronological splits, number of 168-step blocks, metrics tracked (MAE, RMSE, sMAPE)]

Model design

[TODO: Autoformer components used (series decomposition, auto-correlation), input length, size, cited source]

External-data ablation (multi-seed)

Table 6: With vs. without optional external data – mean $\pm$ std over [TODO: k] seeds, validation.

Configuration	MAE	RMSE	sMAPE
Target only			
+ external (past only)			
+ external (past + horizon)			

Final submission

Declared parameters $P =$ [TODO:]; epochs $E =$ [TODO:]; leaderboard score [TODO:].

Reflection questions (optional)

[TODO: Q2.1–Q2.9, if answered]

Appendix – Use of Generative AI

Tool: Claude (Anthropic). For each use: the prompt, the output, and the edits I made.

1. **Prompt:** “help me understand the assignment, and how i need to solve it”
Output: overview of Tasks 1–2 and a plan.
Edits: [TODO: ...]
2. **Prompt:** “write me the code now full for forward”
Output: the 5-line `RawAttentionForecaster.forward` assembly.
Edits: [TODO: ...]
3. **Prompt:** “do it” (report skeleton)
Output: this L^AT_EX template (structure and placeholders only).
Edits: [TODO: ...]
4. **Prompt:** “help me write it” (`SeriesDecomposition.forward`, `aggregate_delays`)
Output: implementations of both functions (replicate padding + `avg_pool1d`; index arithmetic + `gather`).
Edits: [TODO: ...]
5. **Prompt:** “help me write the responses seeing the results”
Output: draft text for Responses 1A (measurement and revision), 1B, 2, 3 and 4(b)–(c), filled tables and figure placement, based on my full-run outputs. My predictions in 1A and 4(a) and my deployment choices were my own.
Edits: [TODO: describe what you changed]
6. [TODO: add further prompts as you go]